

Material de Apoio — Laços de Repetição em C, Python e R

Prof. Bruno Melo · Universidade Positivo — Processo Seletivo Docente

Conteúdo complementar (fora dos 20 min) · guia de consulta

Sumário

A ideia central: a estrutura lógica de um laço — inicializar, testar, executar, atualizar — é a mesma em C, Python e R. Muda a *sintaxe* e o que cada comunidade considera idiomático. Compreender o conceito em C facilita a transferência para as outras linguagens.

1 O que é uma estrutura de repetição

Uma **estrutura de repetição** (laço, *loop*) executa um bloco de comandos várias vezes enquanto uma condição se mantém verdadeira ou enquanto um contador percorre um intervalo (DEITEL; DEITEL, 2011). Distinguimos dois casos:

- **Repetição contada** — o número de iterações é **conhecido** de antemão (naturalmente expressa por `for`).
- **Repetição condicionada** — o número de iterações **depende de uma condição** avaliada em tempo de execução (naturalmente expressa por `while/do-while`) (ASCENCIO; CAMPOS, 2012).

Todo laço tem quatro elementos: **inicialização**, **condição de parada**, **corpo** e **atualização** da variável de controle. A omissão da atualização da variável testada é a causa clássica do **laço infinito** (DEITEL; DEITEL, 2011).

2 O exemplo condutor: simulador de poupança

Problema: o usuário guarda um valor fixo por mês. Quantos meses até atingir uma meta (ex.: comprar um notebook)? E quanto terá em exatamente 12 meses?

2.1 Em C (KERNIGHAN; RITCHIE, 1989)

```
/* Contada: 12 meses fixos */
float total = 0.0f;
for (int mes = 1; mes <= 12; mes++) {
    total += mensal;
    printf("Mes %d: R$ %.2f\n", mes, total);
}

/* Condicionada: ate a meta */
float t = 0.0f; int meses = 0;
while (t < meta) {
    t += mensal;
    meses++;
}
```

2.2 Em Python (DOWNEY, 2016)

```
# Contada: iterar sobre um intervalo (idiomatico em Python)
total = 0.0
for mes in range(1, 13):          # 1..12
    total += mensal
    print(f"Mes {mes}: R$ {total:.2f}")

# Condicionada
t, meses = 0.0, 0
while t < meta:
    t += mensal
    meses += 1
```

Em Python, o `for` percorre os elementos de uma sequência (aqui, `range`) e não um contador manual: iterar sobre coleções é o estilo idiomático (DOWNEY, 2016). Não existe `do-while`; emula-se com `while True: + break`.

2.3 Em R (WICKHAM; GROLEMUND, 2017)

```

# Contada
total <- 0
for (mes in 1:12) {
  total <- total + mensal
  cat(sprintf("Mes %d: R$ %.2f\n", mes, total))
}

# Condicionada
t <- 0; meses <- 0
while (t < meta) {
  t <- t + mensal
  meses <- meses + 1
}

# Estilo idiomático em R: vetorização (evita o laço explícito)
total_12 <- sum(rep(mensal, 12))

```

Em R, laços explícitos funcionam, mas o estilo idiomático prefere **vetorização** e funções de iteração funcional (ex.: `purrr::map`), reservando o `for` para efeitos colaterais (WICKHAM; GROLEMUND, 2017, cap. *Iteration*).

3 Quadro comparativo

Aspecto	C	Python	R
for contado	<code>for(int m=1;m<=12;m++)</code>	<code>for m in range(1,13):</code>	<code>for (m in 1:12)</code>
while	<code>while(cond){}</code>	<code>while cond:</code>	<code>while (cond) {}</code>
do-while	<code>do{}while(cond);</code>	não existe	não existe
Bloco	chaves { }	indentação após :	chaves { }
Controle	tipada, declarada	nome dinâmico	nome dinâmico
Idiomático	contador explícito	iterar coleções	vetorização / <code>map</code>

Conclusão didática: a estrutura lógica (inicializar, testar, executar, atualizar) é a mesma nas três linguagens; muda a sintaxe e o que cada comunidade considera idiomático.

4 Erros comuns (para a depuração ao vivo)

1. **Laço infinito** — a variável testada na condição não é atualizada no corpo. *Ex.:* esquecer `total += mensal`; no `while`; `total` fica 0 e a condição nunca fica falsa (DEITEL; DEITEL, 2011).
2. **Off-by-one** — erro de contagem por um: usar `mes < 12` quando se quer 12 iterações (executa 11), ou iniciar o contador em 0 quando se esperava 1.

3. **Ponto-e-vírgula indevido** — `for (...); com ;` logo após o cabeçalho cria um corpo vazio (armadilha sintática em C).

5 Pergunta de verificação de aprendizagem

No simulador de poupança, quero garantir que o menu apareça pelo menos uma vez e só depois perguntar se o usuário deseja continuar. Qual das três estruturas — `for`, `while` ou `do-while` — é a mais adequada, e por quê?

Resposta esperada: `do-while`. Como a condição é avaliada **ao final** do laço, o corpo (o menu) executa **pelo menos uma vez** antes da primeira verificação — comportamento exigido pelo requisito (DEITEL; DEITEL, 2011).

Questão-desafio (opcional para a banca): “Como obter esse comportamento em Python, que não possui `do-while`?” Resposta: `while True:` com um `break` condicional ao final (DOWNEY, 2016).

6 Referências

ASCENCIO, A. F. G.; CAMPOS, E. A. V. **Fundamentos da programação de computadores:** algoritmos, Pascal, C/C++ (padrão ANSI) e Java. 3. ed. São Paulo: Pearson, 2012.

DEITEL, P.; DEITEL, H. C. **como programar**. 6. ed. São Paulo: Pearson Prentice Hall, 2011.

DOWNEY, A. B. **Pense em Python:** pense como um cientista da computação. 2. ed. São Paulo: Novatec, 2016.

KERNIGHAN, B. W.; RITCHIE, D. M. **C: a linguagem de programação:** padrão ANSI. Rio de Janeiro: Campus, 1989.

MASETTO, M. T. **Competência pedagógica do professor universitário**. 2. ed. São Paulo: Summus, 2012.

WICKHAM, H.; GROLEMUND, G. **R for data science:** import, tidy, transform, visualize, and model data. Sebastopol: O'Reilly Media, 2017.